

Automaten und Sprachen | AutoSpr

Zusammenfassung

INHALTSVERZEICHNIS

1. Vorwort	3
2. Prädikate	3
2.1. Normalformen	3
2.2. Negation	3
3. Mengen	4
3.1. Konstruktion	4
3.2. Operationen	4
3.3. Tupel	4
4. Beweise	4
5. Sprachen	4
5.1. Wortlänge	5
5.2. Deterministische Endliche Automaten (DEA)	6
5.3. Wörter einer regulären Sprache	6
5.3.1. Übergangsfunktionen für Wörter	6
5.3.2. Wort akzeptieren	6
5.3.3. Akzeptierte Sprache, reguläre Sprache	6
5.4. Myhill-Nerode Automat	6
5.5. Minimalautomat	7
5.6. Pumping Lemma	7
5.6.1. Beweis	8
5.7. Nichtdeterministische endliche Automaten (NEA)	8
5.7.1. Umgang mit mehrdeutigen Übergängen	9
5.7.2. Thompson-NEA	9
5.7.3. NEA_ϵ	10
5.8. Mengenoperationen	10
5.9. Reguläre Operationen	11
5.9.1. Alternative	11
5.9.2. Verkettung	12
5.9.3. *-Operation	12
5.10. Reguläre Ausdrücke	12
5.10.1. Verallgemeinerter NEA (VNEA)	12
5.10.2. Teststrategie	14
5.11. Kontextfreie Sprachen	14
5.11.1. Kontextfreie Grammatik und Sprache	14
5.11.2. Reguläre Operationen	14
5.11.3. Kontextfrei	15
5.11.4. Chomsky-Normalform (CNF)	15
6. Parsing	16
6.1. Cocke-Younger-Kasami Algorithmus (CYK)	16
6.1.1. Ableitungsdreieck	17

6.2. Backus-Naur-Form (BNF)	17
6.3. Extended Backus-Naur-Form (EBNF)	18
6.4. Rechtsrekursion	18
7. Stack	19
7.1. Stackautomat / Pushdown Automaton (PDA)	19
7.1.1. Ableitung aus CNF	19
7.1.2. PDA $\rightarrow$ CFG	20
8. Nicht kontextfreie Sprachen	21
8.1. Pumping Lemma	21
9. Unendlich	22
10. Turing-Maschine (TM)	23
10.1. Arbeitsweise	23
10.2. Protokollierung	24
10.3. Varianten	25
10.3.1. Nichtdeterministische TM	26
10.3.2. Verschiedene Bandalphabete	26
10.3.3. Mehrspurmaschine	26
10.3.4. Mehrbandmaschine	26
10.3.5. Einseitig unendliches Band	27
10.4. Turing-erkennbare Sprachen	27
10.4.1. Aufzähler	27
10.4.2. Entscheider und entscheidbare Sprachen	27
11. Entscheidbarkeit	27
11.1. Entscheider	28
11.2. Akzeptanzproblem für Turing-Maschinen	28
11.3. Halteproblem	28
11.4. Reduktion	29
11.5. Vergleich der Entscheidbarkeit von Sprachen	29
11.6. Satz von Rice	29
11.7. Lösungsmethoden für Entscheidungsprobleme	29
12. Komplexität	29
12.1. Hardwareeinfluss	29
12.2. Polynomielle und Exponentielle Laufzeit	30
12.3. Hardwareunabhängige Laufzeit-Komplexität	30
12.4. NP	30
12.5. Polynomielle Verifizierer	30
12.6. Polynomielle Reduktion	30
12.6.1. Polynomieller Laufzeit-Vergleich	30
12.6.2. Satz	30
12.6.3. Reduktion von HAMPATH	31
12.7. Polynomielle Ausfüllrätsel	31
Bibliografie	32

1. VORWORT

Diese Zusammenfassung basiert auf dem Buch «Automaten und Sprachen» [1], geschrieben von Prof. Dr. Andreas Müller.

2. PRÄDIKATE

Prädikate sind Aussagen über mathematische Objekte, die wahr oder falsch sein können. «Funktionen» mit booleschen Rückgabewerten: $P, Q(n), R(x, y, z)$.

$$\begin{aligned} (A \Rightarrow B) &\Leftrightarrow (\neg B \Rightarrow \neg A) & (A \Leftrightarrow B) &\Leftrightarrow (A \wedge B) \vee (\neg A \wedge \neg B) & A \vee (\neg A \wedge B) &\Leftrightarrow A \vee B \\ (A \Rightarrow B) &\Leftrightarrow (\neg A \vee B) & \neg(A \Rightarrow B) &\Leftrightarrow A \wedge \neg B & (A \Rightarrow B \Rightarrow C) &\Leftrightarrow (A \Rightarrow B) \wedge (B \Rightarrow C) \end{aligned}$$

Begriff	Bedeutung
Abtrennungsregel	$(A \wedge (A \Rightarrow B)) \Rightarrow B$
Kommutativität	$(A \wedge B) \Leftrightarrow (B \wedge A)$ $(A \vee B) \Leftrightarrow (B \vee A)$
Assoziativität	$A \wedge (B \wedge C) \Leftrightarrow (A \wedge B) \wedge C$ $A \vee (B \vee C) \Leftrightarrow (A \vee B) \vee C$
Distributivität	$A \wedge (B \vee C) \Leftrightarrow (A \wedge B) \vee (A \wedge C)$ $A \vee (B \wedge C) \Leftrightarrow (A \vee B) \wedge (A \vee C)$
Absorption	$A \vee (A \wedge B) \Leftrightarrow A$ $A \wedge (A \vee B) \Leftrightarrow A$
Idempotenz	$A \vee A = A$ $A \wedge A = A$
Doppelte Negation	$\neg(\neg A) \Leftrightarrow \neg\neg A \Leftrightarrow A$
Konstanten	$W = \text{wahr}$ $F = \text{falsch}$
de Morgan	$\neg(A \wedge B) \Leftrightarrow \neg A \vee \neg B$ $\neg(A \vee B) \Leftrightarrow \neg A \wedge \neg B$

2.1. NORMALFORMEN

Normalformen (allgemein, **kanonische Formen**) helfen, das Vergleichsproblem zu lösen (ob zwei Aussagen dieselben sind).

$$P_1 \wedge \dots \wedge P_n = \forall i \in \{1, \dots, n\} (P_i) = \bigwedge_{i=1}^n P_i$$

$$P_1 \vee \dots \vee P_n = \exists i \in \{1, \dots, n\} (P_i) = \bigvee_{i=1}^n P_i$$

2.2. NEGATION

Nicht für alle = Es gibt einen Fall, für den nicht

$$\neg \forall i \in \{1, \dots, n\} (P_i) \Leftrightarrow \neg (P_1 \wedge \dots \wedge P_n) \Leftrightarrow \neg P_1 \vee \dots \vee \neg P_n \Leftrightarrow \exists i \in \{1, \dots, n\} (\neg P_i)$$

3. MENGEN

3.1. KONSTRUKTION

Grundmenge G , Prädikat $P(x)$. Konstruierte Menge $A = \{x \in G \mid P(x)\}$

3.2. OPERATIONEN

Begriff	Bedeutung
Vereinigung	$A \cup B = \{x \mid x \in A \vee x \in B\}$
Schnittmenge	$A \cap B = \{x \mid x \in A \wedge x \in B\}$
Komplement	$\overline{A} = \{x \mid x \notin A\}$
Differenz	$A \setminus B = \{x \in A \mid x \notin B\}$

3.3. TUPEL

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

n -Tupel:

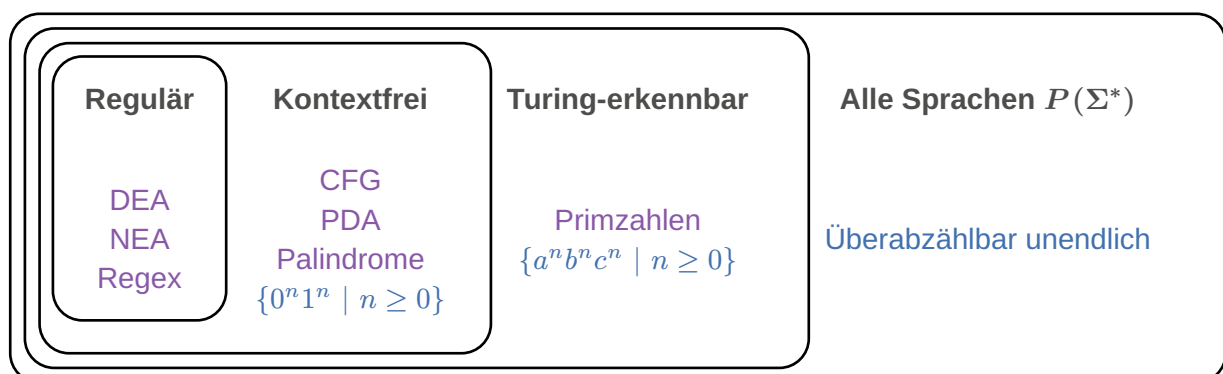
$$\bigtimes_{i=1}^n A_i = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i\}$$

4. BEWEISE

Eine Folge von logischen Schlüssen, die zeigen, dass die Aussage aus den gegebenen Voraussetzungen folgt.

Beweistechnik	Anwendungsfall
Konstruktiver Beweis	Liefert explizite «Lösung» / Algorithmus
Widerspruchsbeweis	Geeignet für Unmöglichkeitssagen. Z.B. $\sqrt{2} \notin \mathbb{Q}$
Vollständige Induktion	Für Aussagen, die für alle $n \in \mathbb{N}$ gelten.

5. SPRACHEN



Begriff	Bedeutung
Alphabet	Eine nichtleere endliche Menge Σ heisst Alphabet . Die Elemente von Σ heissen Zeichen .
Wort	Eine Zeichenkette der Länge n ist ein n -Tupel in $\Sigma^n = \Sigma \times \dots \times \Sigma$. Ein Element von Σ^n heisst Wort der Länge n . Wörter haben immer endliche Länge.
Leeres Wort	Die Zeichenkette $\varepsilon \in \Sigma^0 = \{\varepsilon\}$ der Länge 0 heisst das leere Wort .
Menge aller Wörter	Leeres Wort ist immer drin $\Sigma^* = \{\varepsilon\} \cup \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \bigcup_{k=0}^{\infty} \Sigma^k$
Abgekürzte Schreibweisen von Wörtern	$(A, u, t, o, S, p, r) = AutoSpr$ $a^3b^5 = aaabbbbbb$ $b^5a^3 = bbbbaaaa$
Sprache	Teilmenge $L \subset \Sigma^*$

5.1. WORTLÄNGE

Begriff	Bedeutung
Länge des Wortes	$w \in \Sigma^n \Rightarrow w = n$, n ist Länge des Wortes w
Anzahl Zeichen	Sei $w \in \Sigma^n$, $a \in \Sigma$. Dann ist $ w _a$ die Anzahl Zeichen a im Wort w

Beispiel 1 : Wortlänge

$$\begin{array}{lll}
 |\varepsilon| = 0 & |01010|_1 = 2 & |a^3b^5| = 8 \\
 |01010|_0 = 3 & |01010|_3 = 0 & |(1291)^7| = 7|1291| = 7 \cdot 4 = 28
 \end{array}$$

Beispiel 2 : Sprache

$$\begin{aligned}
 L &= \Sigma^* \subset \Sigma^* \\
 L &= \emptyset \subset \Sigma^*, \text{ die leere Sprache} \\
 \Sigma &= \{0, 1\}, \text{ Sprache aller Binärstrings: } L = \Sigma^* \\
 \Sigma &= \text{Unicode}, J = \{w \in \Sigma^* \mid w \text{ wird vom Java-Compiler akzeptiert}\} \\
 M &\text{ eine "Maschine", } L(M) = \{w \in \Sigma^* \mid w \text{ wird von } M \text{ akzeptiert}\}
 \end{aligned}$$

Beobachtungen 1

- 1.1. $|w^n| = n \cdot |w|$
- 1.2. Zu jeder Maschine M gibt es eine Sprache $L(M)$

5.2. DETERMINISTISCHE ENDLICHE AUTOMATEN (DEA)

Definition

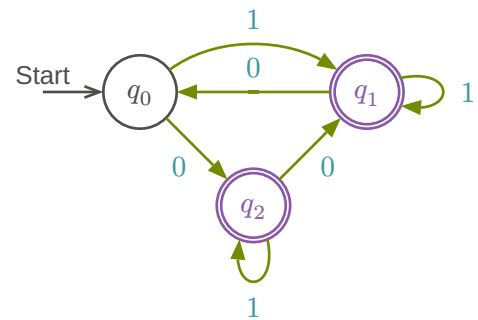
$$A = (Q, \Sigma, \delta, q_0, F)$$

- Zustände: $Q = \{q_0, q_1, \dots, q_n\}$
- Alphabet: Σ
- Übergangsfunktion: $\delta : Q \times \Sigma \rightarrow Q$
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$

Tabellenform

	0	1	Σ
Q	q_2	q_1	
F	q_0	q_1	δ
	q_1	q_2	

Zustandsdiagramm von A



Wichtig: Ein Pfeil für jedes Zeichen in jedem Zustand für validen DEA

5.3. WÖRTER EINER REGULÄREN SPRACHE

5.3.1. Übergangsfunktionen für Wörter

$$\delta : Q \times \Sigma^* \rightarrow Q : (q, w = a_1, \dots, a_n) \mapsto \delta(\dots\delta(\delta(q, a_1), a_2), \dots, a_n)$$

Übergänge ausgehend von q für Zeichen $a_1, \dots, a_n$ nacheinander anwenden.

5.3.2. Wort akzeptieren

Der DEA $A = (\Sigma, Q, q_0, \delta, F)$ **akzeptiert** das Wort $w \in \Sigma^*$, wenn er A von Startzustand in einen Akzeptierzustand $\delta(q_0, w) \in F$ überführt.

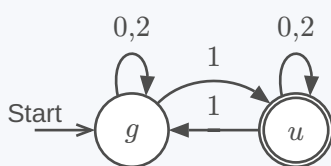
5.3.3. Akzeptierte Sprache, reguläre Sprache

Gegeben ein DEA $A = (\Sigma, Q, q_0, \delta, F)$. Die von A **akzeptierte Sprache** ist

$$L(A) = \{w \in \Sigma^* \mid A \text{ akzeptiert } w\} = \{w \in \Sigma^* \mid \delta(q_0, w) \in F\}$$

Die Sprache $L \subset \Sigma^*$ heisst **regulär**, wenn es einen DEA A gibt mit $L(A) = L$

Beispiel 3: DEA, der die Sprache $L = \{w \in \Sigma^* \mid w \text{ ist eine ungerade Zahl im Dreiersystem}\}$ akzeptiert



	0	1	2
g	g	u	g
u	u	g	u

5.4. MYHILL-NERODE AUTOMAT

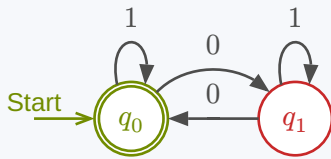
Für ein Wort $w \in \Sigma^*$ setze $L(w) = \{w' \in \Sigma^* \mid ww' \in L\}$ Insbesondere $L(\varepsilon) = L$

Gegeben: reguläre Sprache L über Σ . Rekonstruiere A mit:

- $Q = \{L(w) \mid w \in \Sigma^*\}$
- $q_0 = L(\sigma) = L$
- $F = \{L(w) \in Q \mid \varepsilon \in L(w)\}$
- $\delta(L(w), a) = L(wa)$

Gleicher Zustand $\Leftrightarrow$ gleiches $L(w)$

Beispiel 4: $\Sigma = \{0, 1\}, L = \{w \in \Sigma^* \mid |w|_0 \text{ gerade}\}$



w	$L(w)$	Q
ε	$L(\varepsilon) = L$	q_0
0	$L(0) = \{w \in \Sigma^* \mid w _0 \text{ ungerade}\}$	q_1
1	$L(1) = \{w \in \Sigma^* \mid w _0 \text{ gerade}\}$	q_0
$\vdots$	$\vdots$	$\vdots$

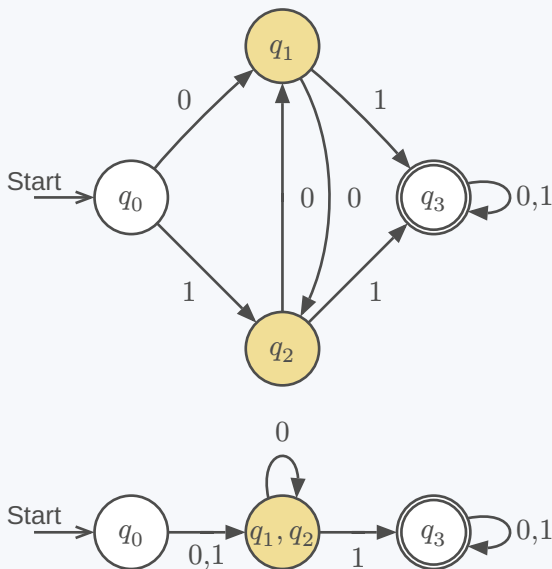
5.5. MINIMALAUTOMAT

Ziel: Nicht unterscheidbare Zustände zusammenlegen

Vorgehen:

- 1) Akzeptierzustand $\neq$ Nichtakzeptierzustand
- 2) Iteration: alle unterscheidbaren Paare suchen
- 3) Verbleibende Paare sind ununterscheidbar $\rightarrow$ zusammenlegen

Beispiel 5



	q_0	q_1	q_2	q_3
q_0	$\equiv$	$\times$	$\times$	$\times$
q_1	$\times$	$\equiv$	$\equiv$	$\times$
q_2	$\times$	$\equiv$	$\equiv$	$\times$
q_3	$\times$	$\times$	$\times$	$\equiv$

Algorithmus

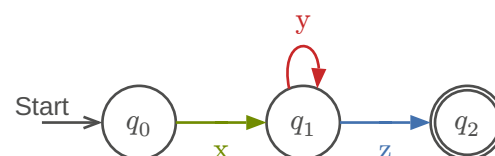
- 1) $q \in F, q' \notin F$
 - 2) Unterscheidbar nach Übergang
 - 3) Iteration bis keine Änderung mehr
- $\Rightarrow q_1$ und q_2 sind nicht unterscheidbar

5.6. PUMPING LEMMA

Ist L eine reguläre Sprache, dann gibt es $N \in \mathbb{N}$, die pumping length so, dass jedes Wort $w \in L$ mit $|w| \geq N$ in drei Teile $w = xyz$ zerlegt werden kann mit:

- 1) $|xy| \leq N$
- 2) $|y| > 0$
- 3) $xy^kz \in L \forall k \in \mathbb{N}$

Oder: genügend lange Wörter ($|w| \geq N$) einer regulären Sprache ($w \in L$) können alle in einem



Anfangsstück der Länge N ($|xy| \leq N$) aufgepumpt werden ($xy^kz \in L$).

Was zeichnet reguläre Sprachen aus?

- Reguläre Ausdrücke
- Lange Wörter: * kommt im regulären Ausdruck vor
- Blöcke mit * können beliebig oft wiederholt werden
- $\Rightarrow$ Wörter können «aufgepumpt» werden

5.6.1. Beweis

Behauptung: Die Sprache $L \subset \Sigma^*$ ist nicht regulär.

Beweis (Widerspruch):

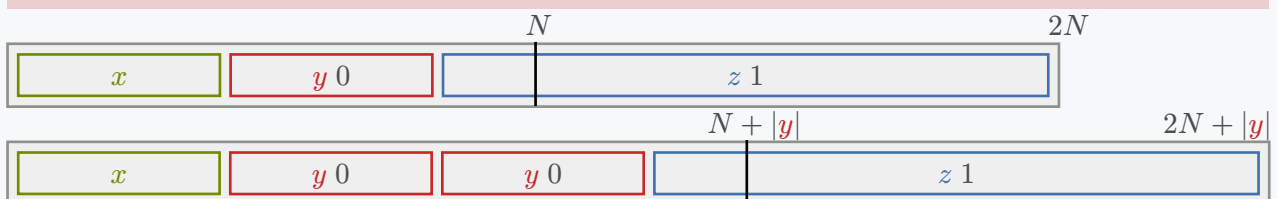
- 1) Annahme: L ist regulär
- 2) Gemäss Pumping Lemma gibt es die Pumping Length N
 - Es darf keine Annahme über die konkrete Grösse von N gemacht werden!
- 3) Wähle ein Wort $w \in L$ mit $|w| \geq N$
 - Die Definition **muss** N verwenden!
- 4) Aufteilung des Wortes gemäss Pumping Lemma
 - $w = xyz, |xy| \leq N, |y| > 0$
- 5) Auswirkung des Pumpens
 - $xy^kz \notin L$ für mindestens ein $k \in \mathbb{N}$ (mit Begründung!)
- 6) Widerspruch und Schlussfolgerung, dass die Annahme nicht zutreffen kann

Beispiel 6 : $L = \{0^n 1^n \mid n \geq 0\}$ ist nicht regulär

- 1) Annahme: L ist regulär
- 2) $\exists N \in \mathbb{N}$, Pumping Length
- 3) $w = 0^N 1^N$
- 4) Unterteilung $w = xyz$

TODO:

reduce confusion



- 5) Pumpen: nur die Anzahl der 1 wird erhöht, Anzahl 0 bleibt
- 6) $xy^kz \notin L$ für $k \neq 1$, im Widerspruch zum Pumping Lemma

5.7. NICHTDETERMINISTISCHE ENDLICHE AUTOMATEN (NEA)

Definition

$$A = (Q, \Sigma, \delta, q_0, F)$$

- Endliche Menge von Zuständen: Q
- Alphabet: Σ
- Übergangsfunktion: $\delta : Q \times \Sigma \rightarrow P(Q)$

Akzeptieren

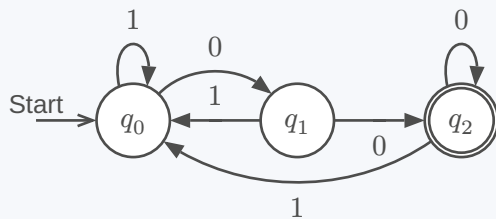
Ein NEA A **akzeptiert** das Wort $w \in \Sigma^*$, wenn es eine **Wahl** von Übergängen gibt, derart, dass das Wort w den Automaten in einen Akzeptierzustand überführt.

Faustregeln

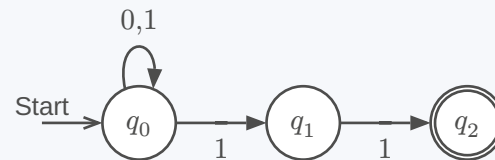
- Startzustand: $q_0 \in Q$
- Akzeptierzustände: $F \subset Q$
- Nur genau diejenigen Pfeile einzeichnen, die man zum Akzeptieren braucht.
- Erlaube Alternativen

Beispiel 7: $L = \{w \in \Sigma^* \mid w \text{ endet mit zwei } 0\}$

DEA-Lösung



NEA-Lösung

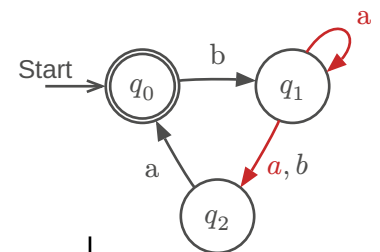


5.7.1. Umgang mit mehrdeutigen Übergängen

a-Übergang von q_1 aus nicht eindeutig

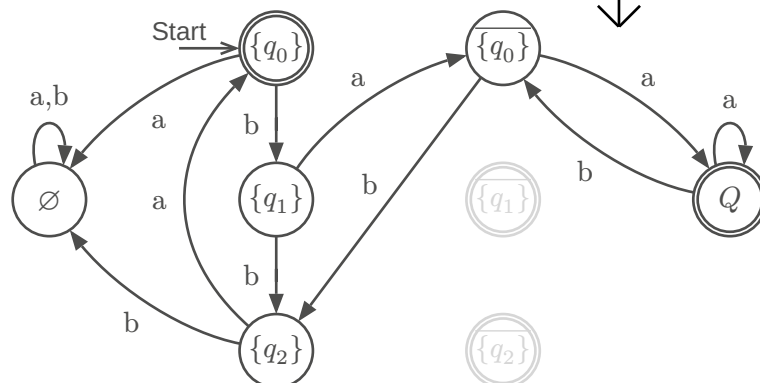
Welchen Übergang soll man nehmen?

- Beide Möglichkeiten probieren und akzeptieren, falls eine der Möglichkeiten auf einen Akzeptierzustand führt.
- Zufallsgenerator (probabilistischer endlicher Automat, PEA)



5.7.2. Thompson-NEA

- Zustand = **Menge** der möglichen Zustände
- Akzeptieren = Ob NEA im Zustand sein **könnte**



5.7.2.1. Transformation NEA → DEA

Gegeben $\delta : Q \times \Sigma \rightarrow P(Q)$ eines NEA. Übergangsfunktion für Mengen $M \subset Q$

$$\delta' : P(Q) \times \Sigma \rightarrow P(Q) : (M, a) \mapsto \delta'(M, a) = \bigcup_{q \in M} \delta(q, a)$$

Satz

Ein NEA A kann in einen DEA A' umgewandelt werden mit

- $Q' = P(Q)$
- $\Sigma' = \Sigma$
- δ' wie oben definiert
- $q'_0 = \{q_0\}$
- $F' = \{M \in P(Q) \mid F \cap M \neq \emptyset\}$

Implementation

Thompson-NEA:

- Zustand $M \subset Q$ realisiert durch Markierung der Zustände in M
- δ' realisiert durch Verschieben von Markierungen
- Akzeptieren: mindestens ein Akzeptierzustand markiert

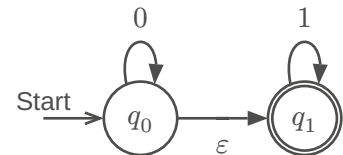
5.7.3. NEA_ϵ

Begriff	Bedeutung
ϵ -Übergang	Kann ohne Verarbeitung eines Zeichens genommen werden. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$
NEA_ϵ	NEA mit ϵ -Übergängen

Einen NEA_ϵ kann man immer in einen NEA umwandeln. Beweis:

$$L = \{0^k 1^l \mid k, l \geq 0\}$$

- 1) $E(q)$ = Menge der von q aus mit ϵ -Übergängen erreichbaren Zustände
- 2) $E(M) = \bigcup_{q \in M} E(q)$
- 3) δ ersetzen durch $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q) : (q, a) \mapsto E(\delta(q, a))$

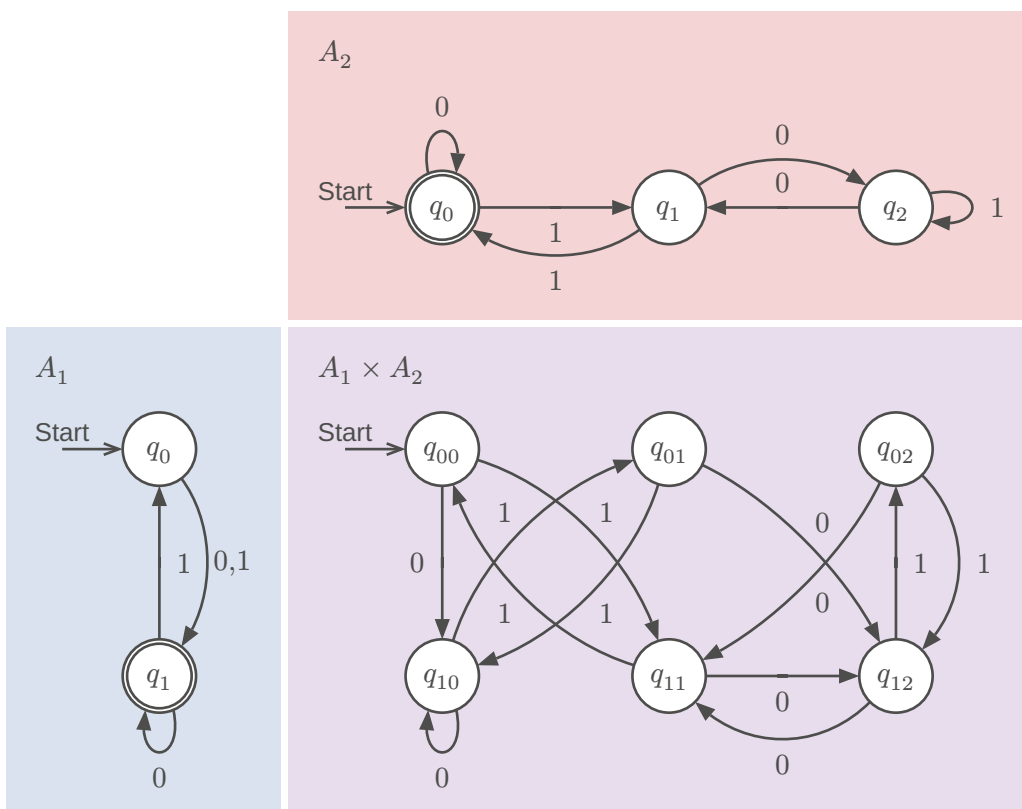


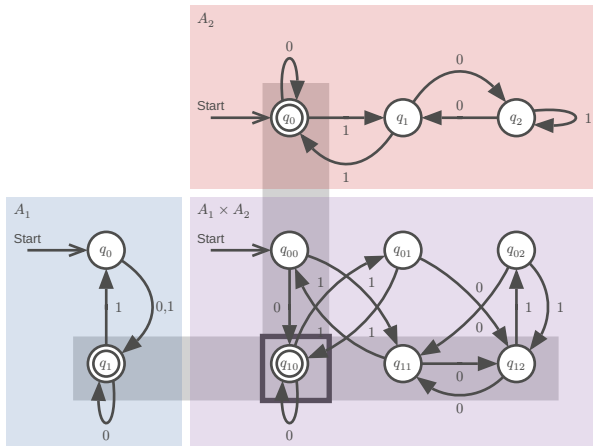
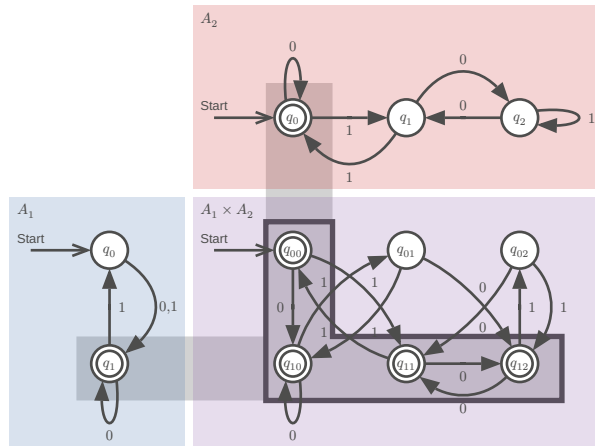
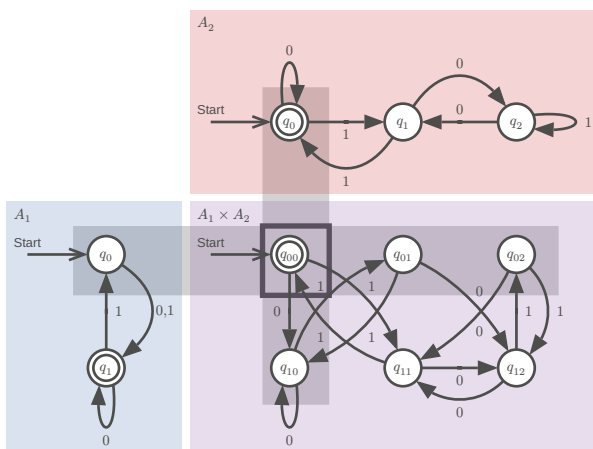
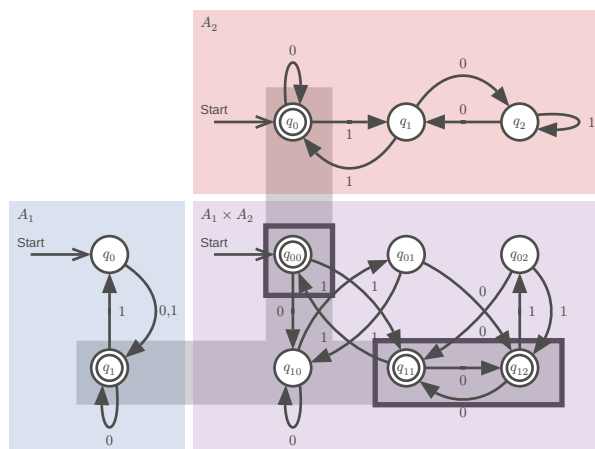
5.8. MENGENOPERATIONEN

Lassen reguläre Sprachen regulär sein.

Produktautomat: $L_i = L(A_i)$

$$L = \{w \in \Sigma^* \mid |w|_1 \text{ ungerade}\} \cap \{w \in \Sigma^* \mid w \text{ ist eine durch drei teilbare Binärzahl}\}$$



Schnittmenge $L(A_1) \cap L(A_2)$ Vereinigungsmenge $L(A_1) \cup L(A_2)$ Differenzmenge $L(A_1) \setminus L(A_2)$ Symmetrische Differenz $L(A_1) \triangle L(A_2)$ 

Operationen verändern nur Endzustände:

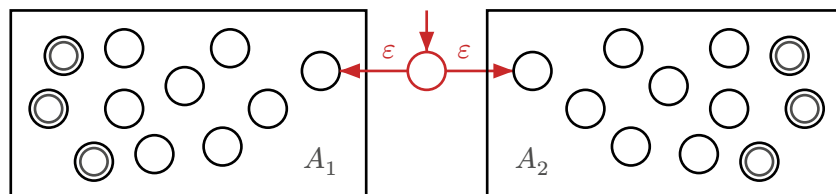
Begriff	Bedeutung
Schnittmenge $L_1 \cap L_2$	$F = F_1 \times F_2$
Vereinigung $L_1 \cup L_2$	$F = F_1 \times Q_2 \cup Q_1 \times F_2$
Differenz $L_1 \setminus L_2$	$F = F_1 \times (Q_2 \setminus F_2)$

5.9. REGULÄRE OPERATIONEN

WICHTIG: ϵ -Übergänge immer hinzufügen

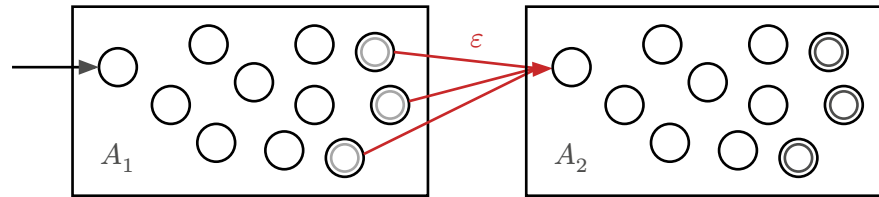
5.9.1. Alternative

$$\begin{aligned}
 L &= L_1 \cup L_2 \\
 &= L(A_1) \cup L(A_2) \\
 &= L(A_1) \mid L(A_2)
 \end{aligned}$$



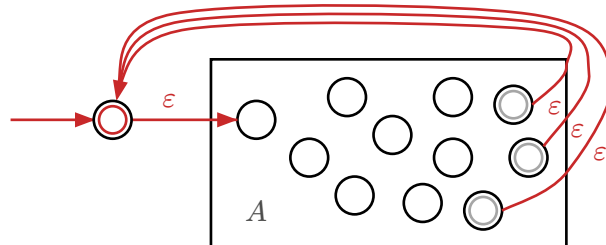
5.9.2. Verkettung

$$\begin{aligned}
 L &= L_1 L_2 \\
 &= L(A_1) L(A_2) \\
 &= \{w_1 w_2 \mid w_i \in L_i\}
 \end{aligned}$$



5.9.3. *-Operation

$$\begin{aligned}
 L^* &= \{\varepsilon\} \cup L \cup L^2 \cup \dots \\
 &= \bigcup_{k=0}^{\infty} L^k
 \end{aligned}$$



⇒ die Klasse der regulären Sprachen ist abgeschlossen unter regulären Operationen

5.10. REGULÄRE AUSDRÜCKE

Formeln, die Zeichenketten beschreiben.

- 1) Buchstaben stehen für sich selbst, mit Ausnahme der Metazeichen $()[]^*? | . \backslash$ (escape character)
- 2) Verkettung: Zeichen und Formeln hintereinanderschreiben
- 3) $.$ = ein beliebiges Zeichen, Σ
- 4) $|$: Alternative, $a|A = a$ oder A
- 5) $*$: Wiederholung, beliebige viele
- 6) $()$: Gruppierung
- 7) $[]$ Zeichenklassen: $[abc] = a|b|c$, $[\wedge abc] =$ alles ausser a , b , oder c

Erweiterungen / Dialekte

- 1) $\{n, m\}$: zwischen n und m Wiederholungen
- 2) $+$: mindestens eines, $\{1, \infty\}$
- 3) $?$: optional, $\{0, 1\}$
- 4) Symbole für Zeichenklassen: $\backslash d$ Ziffern, $\backslash s$ whitespace, $[: space :]$, $[: lower :]$

5.10.1. Verallgemeinerter NEA (VNEA)

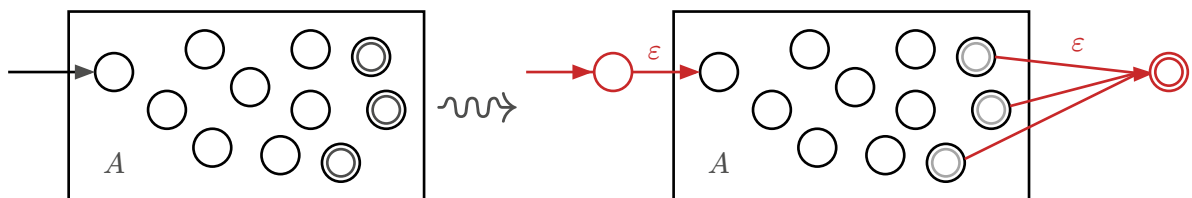
Begriff	Bedeutung
Regulärer Ausdruck	Zeichenkette r zur Beschreibung einer regulären Sprache $L = L(r)$
Reguläre Operationen	$L(r_1) \cup L(r_2) = L(r_1 r_2)$ $L(r_1) L(r_2) = L(r_1 r_2)$ $L(r_1)^* = L(r_1^*)$
Verallgemeinerter NEA	NEA_ε , dessen Übergänge mit regulären Ausdrücken beschriftet sind

Begriff	Bedeutung	
Primitive reguläre Ausdrücke	Reguläre Ausdrücke für Wörter mit Länge ≤ 1	
$L = L(r)$	r	NEA
$\emptyset$	$\emptyset$	Start $\rightarrow$
$\{\varepsilon\}$	$\{\varepsilon\}$	Start $\rightarrow$
$\{a\}$	a	Start $\rightarrow$
$\{o, s, t\}$	$[ost]$	Start $\rightarrow$
$\{a, b, \dots, s\}$	$[a-s]$	Start $\rightarrow$
Σ	$.$	Start $\rightarrow$

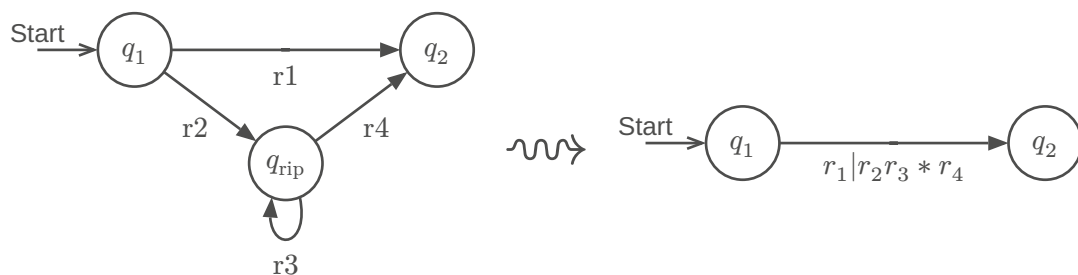
$\Rightarrow$ zu jedem regulären Ausdruck gibt es einen DEA

5.10.1.1. VNEA A umwandeln in Regex r

Keine Übergänge nach q_0 und nur ein Akzeptierzustand



Reduktion



Regulärer Ausdruck

Nach Entfernen aller Zwischenzustände $q_{\text{rip}} \in Q$ bleibt ein regulärer Ausdruck r von A



$\Rightarrow$ jede reguläre Sprache lässt sich mit einem regulären Ausdruck beschreiben $\{L(A) \mid A \text{ ein DEA}\} = \{L(r) \mid r \text{ ein regulärer Ausdruck}\}$

Interessantes projekt (DEA Lexer DSL): [Ragel](#)

5.10.2. Teststrategie

Messung	Folgerung
Für $n = 1, 2, 3, \dots$	– Laufzeit $O(2^n)$: NEA-Implementation
– Regulären Ausdruck erzeugen $r = \underbrace{a? a? a? a? \dots a?}_{n \cdot a?} \underbrace{aaaa \dots a}_{n \cdot a}$	– Laufzeit $O(n)$: DEA-Implementation
– Laufzeit für Akzeptieren von a^n durch r messen	

5.11. KONTEXTFREIE SPRACHEN

Begriff	Bedeutung
CFL	Context Free Language
CFG	Context Free Grammar
PDA	Pushdown Automata

5.11.1. Kontextfreie Grammatik und Sprache

Kontextfreie Grammatik: $G = (V, \Sigma, R, S)$

- V : Variablen
- Σ : Terminalsymbole (Alphabet)
- R : Regeln der Form $A \rightarrow x_1 x_2 \dots x_n$ mit $A \in V, x_i \in V \cup \Sigma$
- S : Startvariable

Ableitung und erzeugte Sprache

- Regel $A \rightarrow w$ erzeugt aus uAv das Wort uwv : $uAv \Rightarrow uwv$
- v aus u ableiten: $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \dots \Rightarrow u_n \Rightarrow v$ oder $u \xRightarrow{*} v$
- $L(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}$ von G erzeugte kontextfreie Sprache

Erzeugte Sprache: Die Menge der Wörter, die von einer kontextfreien Grammatik G aus der Startvariablen abgeleitet werden können, wird mit $L(G)$ bezeichnet und heißt die von G erzeugte Sprache.

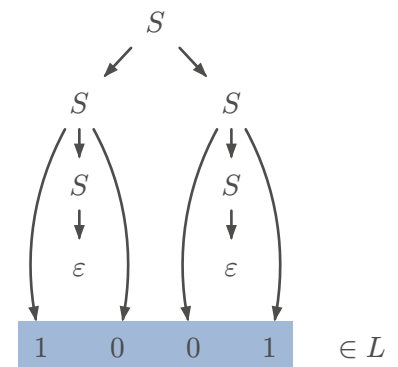
Kontextfreie Sprache: Eine Sprache L heißt kontextfrei, wenn es eine kontextfreie Grammatik gibt, die die Sprache $L = L(G)$ erzeugt.

Parse Tree

$$L = \{w \in \{0, 1\}^* \mid |w|_0 = |w|_1\}$$

mit der Grammatik

$$S \rightarrow 0S1 \mid 1S0 \mid SS \mid \varepsilon$$



Beispiel 8: $L = \{0^n 1^n \mid n \geq 0\}$

Grammatik $L = \{0^n 1^n \mid n \geq 0\}$

- 1) Variablen: $V = \{S\}$
- 2) Terminalsymbole: $\Sigma = \{0, 1\}$
- 3) Regeln: $R = \{S \rightarrow \varepsilon \mid 0S1\}$
- 4) Startvariable: S

Wörter, die erzeugt werden müssen

$$\begin{aligned} \varepsilon \\ 01 = 0\varepsilon 1 \\ 0011 = 0011 \end{aligned}$$

Es ist nicht garantiert, dass es für die Ableitung eines Wortes nur einen einzigen Syntaxbaum gibt. Man sagt, die Grammatik ist **nicht eindeutig**, wenn sie mehrere Syntaxbäume für die gleichen Wörter erlaubt.

5.11.2. Reguläre Operationen

Die Klasse der kontextfreien Sprachen ist abgeschlossen unter regulären Operationen.

Reguläre Sprachen sind kontextfrei.

5.11.2.1. Grammatik für reguläre Operationen

Seien L_1 und L_2 kontextfreie Sprachen mit Grammatiken $G_i = (V_i, \Sigma, R_i, S_i)$. Wir nehmen an, dass die Variablen- und Regelmengen disjunkt sind, also $V_1 \cap V_2 = \emptyset \wedge R_1 \cap R_2 = \emptyset$. Zudem sei S_0 eine Variable, die nicht in $V_1 \cup V_2$ enthalten ist. Dann gilt:

Alternative $L_1 \mid L_2$ ist kontextfrei mit der Grammatik

$$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 \mid S_2\}, S_0)$$

Verkettung $L_1 L_2$ ist kontextfrei mit der Grammatik

$$G = (V_1 \cup V_2 \cup \{S_0\}, \Sigma, R = R_1 \cup R_2 \cup \{S_0 \rightarrow S_1 S_2\}, S_0)$$

-Operation L_1^ ist kontextfrei mit der Grammatik

$$G = (V_1 \cup \{S_0\}, \Sigma, R = R_1 \cup \{S_0 \rightarrow S_0 S_1, S_0 \rightarrow \varepsilon\}, S_0)$$

5.11.3. Kontextfrei

5.11.3.1. Regeln ohne Kontext

Es spielt keine Rolle, in welchem Kontext das Zeichen A vorkommt, die Regel kann immer angewendet werden

$$S \rightarrow aAb \rightarrow aab$$

$$S \rightarrow A \mid C$$

$$A \rightarrow a$$

$$A \rightarrow aAb$$

$$C \rightarrow bA$$

5.11.3.2. Regeln mit Kontext

Je nach **Kontext** kann eine Regel nicht unbedingt angewendet werden

$$S \rightarrow aC \rightarrow A$$

$$S \rightarrow bC \rightarrow B$$

$$S \rightarrow C \rightarrow ?$$

$$S \rightarrow C \mid aC \mid bC$$

$$aC \rightarrow A$$

$$bC \rightarrow B$$

5.11.4. Chomsky-Normalform (CNF)

Eine CFG ist in **Chomsky-Normalform**, wenn S auf der rechten Seite nicht vorkommt und jede Regel von der Form $A \rightarrow BC$ oder $A \rightarrow a$ ist, zusätzlich ist die Regel $S \rightarrow \varepsilon$ erlaubt.

5.11.4.1. Umwandlung

- 1) Neue Startvariable $S_0 \rightarrow S$ (wenn nötig)
- 2) ε -Regeln: $\left. \begin{matrix} A \rightarrow \varepsilon \\ B \rightarrow AC \end{matrix} \right\} \Rightarrow A$ kann weggelassen werden $\Rightarrow \left\{ \begin{matrix} B \rightarrow AC \\ \rightarrow AC \end{matrix} \right.$
- 3) Unit-Rules: $\left. \begin{matrix} A \rightarrow B \\ B \rightarrow CD \end{matrix} \right\} \Rightarrow$ aus A kann man wie aus B auch CD machen $\Rightarrow \left\{ \begin{matrix} A \rightarrow CD \\ B \rightarrow CD \end{matrix} \right.$
- 4) Verkettungen: $A \rightarrow u_1 u_2 \dots u_n$ ersetzen durch $A \rightarrow u_1 A_1, A_1 \rightarrow u_2 A_2, \dots, A_{n-2} \rightarrow u_{n-1} u_n$ und falls u_i ein Terminalsymbol ist: $A_{i-1} \rightarrow U_i A_i, U_i \rightarrow u_i$.

5.11.4.2. Folgerungen

- 1) Ableitung eines Wortes $w \in L(G)$ ist immer in $2|w| - 1$ Regelanwendungen möglich. Beweis:
 - $|w| - 1$ Regeln der Form $A \rightarrow BC$ um aus S ein Wort aus $|w|$ Variablen zu erzeugen
 - $|w|$ Regeln der Form $A \rightarrow a$ um das Wort w zu erzeugen
 - $\Rightarrow$ insgesamt $2|w| - 1$ Regelanwendungen
- 2) Deterministischer Parse-Algorithmus mit Laufzeit $O(|w|^3)$

Beispiel 9 : $S \rightarrow ASA \mid aB$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \varepsilon$$

1) Startvariable

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid aB$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b \mid \varepsilon$$

2) ε -Regel

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid aB \mid a$$

$$A \rightarrow B \mid \varepsilon \mid S$$

$$B \rightarrow b$$

$$S_0 \rightarrow S$$

$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$

$$A \rightarrow B \mid S$$

$$B \rightarrow b$$

3) Unit-Regel

$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$

$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$

$$A \rightarrow B \mid ASA \mid SA \mid AS \mid aB \mid a$$

$$B \rightarrow b$$

$$S_0 \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$

$$S \rightarrow ASA \mid SA \mid AS \mid aB \mid a$$

$$A \rightarrow b \mid ASA \mid SA \mid AS \mid aB \mid a$$

$$B \rightarrow b$$

4) Komplexe Regeln

$$S_0 \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$

$$S \rightarrow AA_1 \mid SA \mid AS \mid UB \mid a$$

$$A \rightarrow b \mid AA_1 \mid SA \mid AS \mid UB \mid a$$

$$A_1 \rightarrow SA$$

$$U \rightarrow a$$

$$B \rightarrow b$$

6. PARSING

6.1. COCKE-YOUNGER-KASAMI ALGORITHMUS (CYK)

Gegeben

1) Grammatik $G = (V, \Sigma, R, S)$ 2) Variable $A \in V$ 3) Wort $w \in \Sigma^*$

Frage

Ist w ableitbar? Formell geschrieben: $A \xRightarrow{*} w$ – Spezialfall $w = \varepsilon$: $A \xRightarrow{*} \varepsilon \Leftrightarrow A \rightarrow \varepsilon \in R$ (Epsilonregel, $A = S$)– Spezialfall $|w| = 1$: $A \xRightarrow{*} w \Leftrightarrow A \rightarrow w \in R$ (Terminalsymbolregel)– Fall $|w| > 1$:

$$A \xRightarrow{*} w \Rightarrow \exists \begin{cases} A \rightarrow BC \\ w = w_1 w_2 \end{cases} \in R \quad \text{mit} \quad \begin{cases} B \xRightarrow{*} w_1 \\ C \xRightarrow{*} w_2 \end{cases}$$

6.1.1. Ableitungsdreieck

Ideen

- Parse Tree aus $A \rightarrow BC$ und $T \rightarrow t$
- Variablen in Tabelle füllen

Prinzip

- Einem Teilwort entspricht ein Feld der Tabelle (rot hinterlegt)
- Das Feld wird mit den Variablen gefüllt, aus denen das Teilwort abgeleitet werden kann.

Beispiel

Parse des Wortes $() [()]$

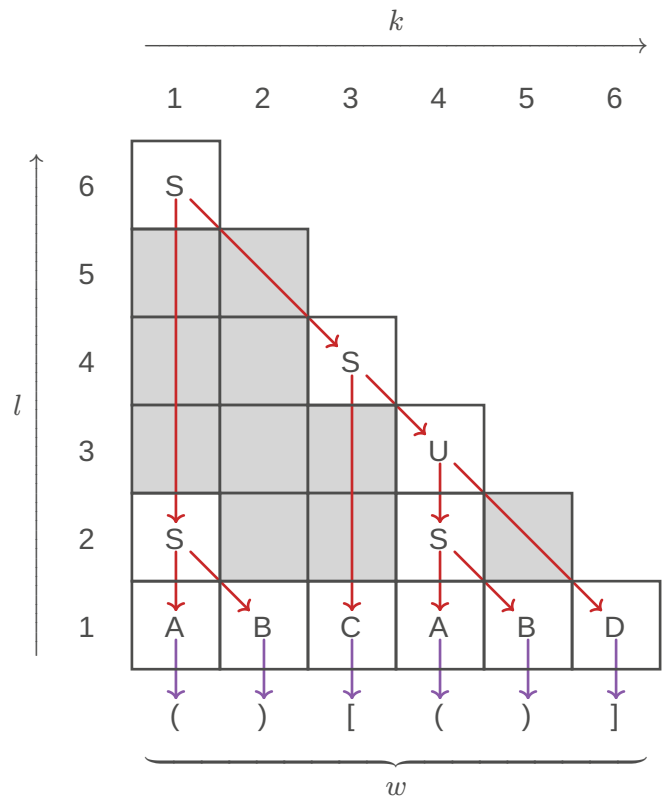
$S \rightarrow AB \mid CD \mid CU \mid SS$ $B \rightarrow)$

$U \rightarrow SD$ $C \rightarrow [$

$A \rightarrow ($ $D \rightarrow]$

Definitionen

Die Felder (k, l_1) und $(k + l_1, l - l_1)$ heissen **korrespondierende Felder** im Ableitungsdreieck des Feldes (k, l) .



TODO:

rekursion vs iteration (buch 149)

6.2. BACKUS-NAUR-FORM (BNF)

Spezifikation der Regeln in maschinen-lesbarer Form:

- Variablen: **<variablen-name>**
- Einzelne Zeichen: A
- Zeichenketten: 'BEISPIEL'
- Regeln: **<variablen-name> ::= Ausdruck**
- Ausdrücke sind Folgen von Variablen, einzelnen Zeichen oder Zeichenketten, getrennt durch |

Beispiel 10 : BNF für Expression-Term-Factor Grammatik

Ausgangsgrammatik

expression $\rightarrow$ expression + term | term

term $\rightarrow$ term * factor | factor

factor $\rightarrow$ (expression) | N

N $\rightarrow$ NZ | Z

Z $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Backus-Naur-Form

```

<expression> ::= <expression> + <term> | <term>
<term>       ::= <term> * <factor> | <factor>
<factor>     ::= ( <expression> ) | <number>
<number>     ::= <number> <digit> | <digit>
<digit>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

```

6.3. EXTENDED BACKUS-NAUR-FORM (EBNF)**Definition**

- Literale in Anführungszeichen oder Apostroph
- = statt :: =
- Variablen ohne < und >, dürfen Leerzeichen enthalten
- Komma für Verkettung ,
- Regel-Endzeichen ;
- Kommentare (* ... *)
- Optionale Wiederholung { ... }
- Option [...]
- Gruppierung (...)
- Ausnahme: -

Achtung!

- Keine Operatorpriorisierung
- Zweideutig!
- Keine eindeutige Evaluation!

Beispiel 11 : EBNF für Expression-Term-Factor Grammatik

```

expression = expression, "+", term | term ;
term       = term, "*", factor, | factor ;
factor     = "(", expression, ")" | number ;
number     = digit, { digit } ;
digit      = "0" | "1" | "2" | "3" | "4" |
            "5" | "6" | "7" | "8" | "9" ;

```

6.4. RECHTSREKURSION

Expression-Term-Factor-Grammatik verwendet Links-Rekursion $\Rightarrow$ korrekte Auswertung von links nach rechts. Parsetree wertet aber Ausdrücke von rechts nach links aus! Alternative Expression-Term-Factor definition mit Rechtsrekursion statt Linksrekursion:

$$\begin{aligned}
 \text{expression} &\rightarrow \text{term expression}' \\
 \text{expression}' &\rightarrow + \text{term expression}' \mid \varepsilon \\
 \text{term} &\rightarrow \text{factor term}' \\
 \text{term}' &\rightarrow * \text{factor term}' \mid \varepsilon \\
 \text{factor} &\rightarrow (\text{expression}) \mid N \\
 N &\rightarrow NZ \mid Z \\
 Z &\rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
 \end{aligned}$$

TODO:

grid → table

7. STACK

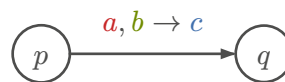
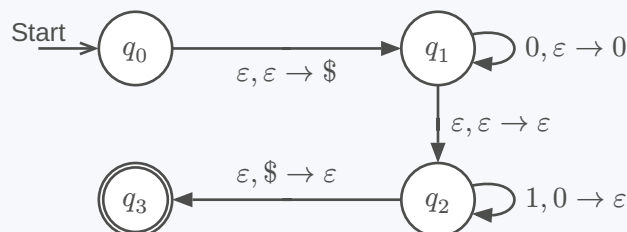
Unendlich grosser Speicher

- Immer nur das oberste Element sichtbar
- Beliebig tief

7.1. STACKAUTOMAT / PUSHDOWN AUTOMATON (PDA)

Kann Sprachen akzeptieren, die nicht regulär sind, ist aber nicht deterministisch. **Beachte:** $\Gamma \neq \Sigma$ möglich**Definition**Stackautomat $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$

- 1) Q : Zustände
- 2) Σ : Eingabe-Alphabet
- 3) Γ : Stack-Alphabet
- 4) $\delta: Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow P(Q \times \Gamma_\epsilon)$
- 5) $q_0 \in Q$: Startzustand
- 6) $F \subset Q$: Akzeptierzustände

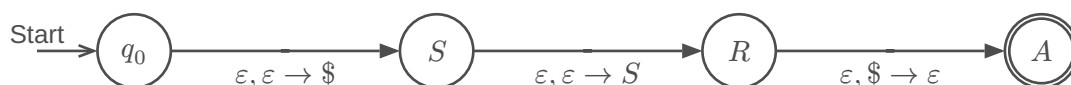
Übergänge a vom Input b vom Stack entfernen (Bedingung) c auf den Stack**Beispiel 12 :** Stackautomat für die Sprache $\{0^n 1^n \mid n \geq 0\}$ **TODO:**

Book 167-168, shift/reduce

TODO:

diagrams? (slides 8/12)

7.1.1. Ableitung aus CNF

Ist L eine kontextfreie Sprache, dann gibt es einen Stackautomaten P , der L akzeptiert, $L = L(P)$.**Grundgerüst**

Regel $A \rightarrow BC$	Regel $A \rightarrow a$	Regel $S \rightarrow \varepsilon$	$\forall a \in \Sigma$

7.1.2. PDA $\rightarrow$ CFG

Variablen

Wörter beschreiben Pfade durch den Automaten: Variable A_{pq} = Wörter, die von p nach q führen mit leerem Stack

Regeln

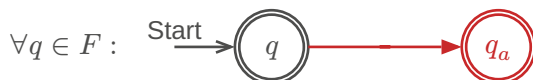
Regeln beschreiben, wie sich Wege zerlegen lassen: $A_{pq} \rightarrow A_{pr}A_{rq}$ = Wege von p nach q können verstanden werden als Wege von p nach r und von dort nach q

7.1.2.1. Stackautomat standardisieren

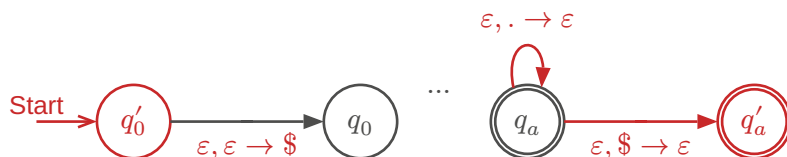
TODO:

better diagrams, book 181

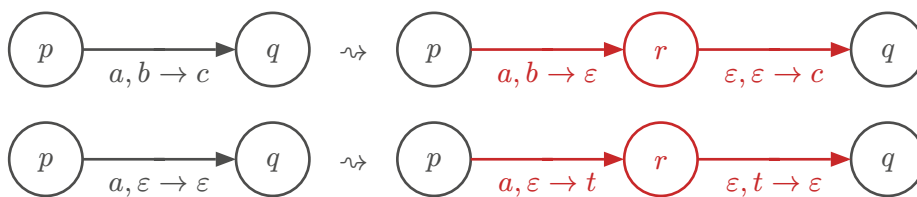
- 1) Nur ein Akzeptierzustand: Neuen Akzeptierzustand q_a und Übergänge von allen vorherigen Endzuständen zu q_a erstellen.



- 2) Stack leeren: Mit einem Element $\$$ erweitern, sodass garantiert werden kann, dass der Stack am schluss leer ist.



- 3) Jeder Übergang legt entweder ein Zeichen auf den Stack oder entfernt eines



TODO:

Book 182,183

Beispiel 13

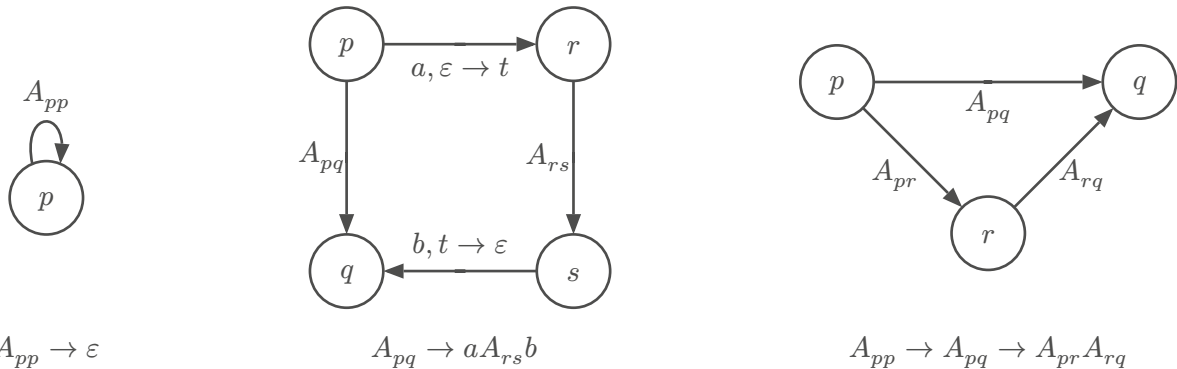
TODO:

visualization

7.1.2.1.1. Grammatik ablesen

Ausgangspunkt: standardisierter PDA mit Startzustand q_0 und $F = \{q_a\}$.

- 1) Startvariable: A_{q_0, q_a}
- 2) Regeln:



Beispiel 14

TODO:

8. NICHT KONTEXTFREIE SPRACHEN

8.1. PUMPING LEMMA

Pumping Lemma für CFL

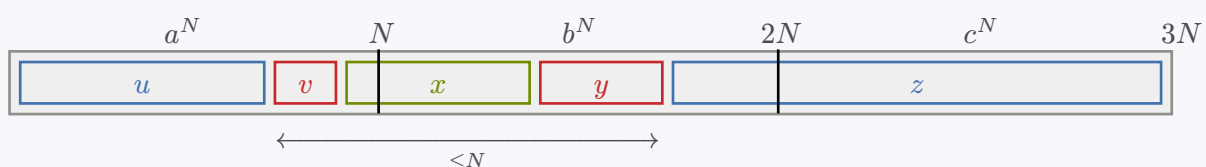
Ist L eine CFL, dann gibt es eine Zahl N , die Pumping Length, derart, dass jedes Wort $w \in L$ mit $|w| \geq N$ zerlegt werden kann in fünf Teile $w = uvxyz$ derart, dass

- 1) $|vy| > 0$
- 2) $|vxy| \leq N$
- 3) $uv^kxy^kz \in L \forall k \in \mathbb{N}$

Mit dem Pumping Lemma kann man beweisen, dass eine Sprache **nicht** kontextfrei ist.

Beispiel 15 : $\{a^n b^n c^n \mid n \geq 0\}$

- 1) Annahme: L kontextfrei
- 2) Pumping length N
- 3) Wort: $w = a^N b^N c^N$
- 4) Zerlegungen:



- 5) Beim Pumpen nimmt die Anzahl der a und b zu, nicht aber die Anzahl der c
 $\Rightarrow uv^kxy^kz \notin L \forall k \neq 1$
- 6) Widerspruch: L nicht kontextfrei

8.1.0.1. Herleitung

Grammatik G in CNF

$$w \in L(G) \Rightarrow S \xRightarrow{*} w$$

Wiederverwendete Variable

$|w| \geq N$ gross genug $\Rightarrow$ Variablen werden im Parse Tree wiederverwendet Die «unterste» wiederverwendete Variable A erzeugt zwei Wörter:

$$\begin{aligned} A &\xRightarrow{*} vxy \\ A &\xRightarrow{*} x \end{aligned}$$

Pumpen

$A \xRightarrow{*} vxy$ anstelle des «untersten» $A \xRightarrow{*} x$ verwenden

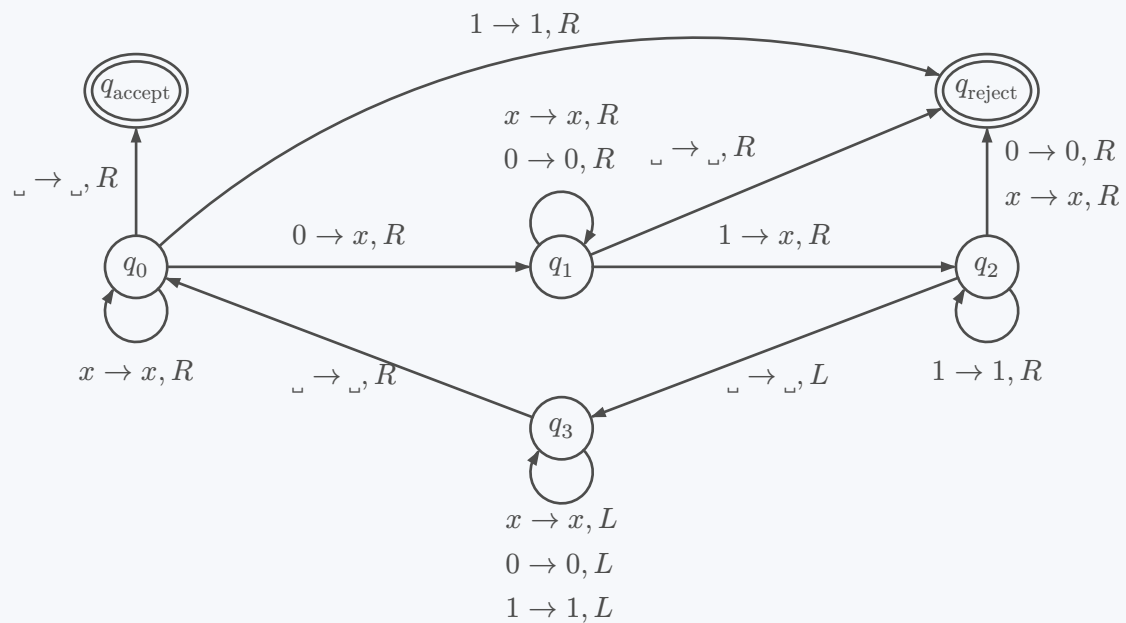
TODO:
diagram

9. UNENDLICH

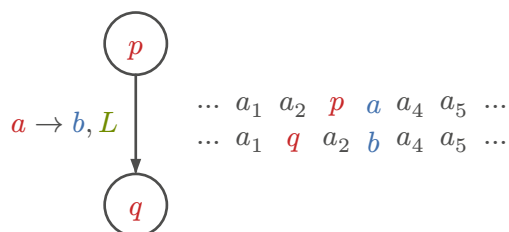
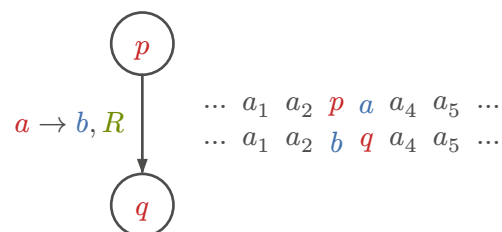
Begriff	Bedeutung
Endlich	Eine Menge A heisst endlich , wenn jede Injektion $A \rightarrow A$ auch eine Bijektion ist
Abzählbar unendlich	Eine Menge A heisst abzählbar unendlich , wenn es eine Bijektion $\mathbb{N} \rightarrow A$ gibt, also $A \simeq \mathbb{N}$. Bsp: $\mathbb{R}, \mathbb{Z}, \mathbb{Q}, \Sigma^*$, Menge aller DEAs/NEAs/PDAs/CFGs
Überabzählbar unendlich	Eine Menge A heisst überabzählbar unendlich , wenn sie nicht abzählbar unendlich ist. Bsp: $\mathbb{C}, P(\mathbb{N})$, Menge aller Sprachen $P(\Sigma^*)$
Gleich mächtig	Mengen A und B heissen gleich mächtig , $A \simeq B$, wenn es eine Bijektion $A \rightarrow B$ gibt
Unendlich	Eine Menge A heisst unendlich , wenn sie gleich mächtig wie eine Teilmenge ist

A, B, A_k abzählbar unendlich, $k \in \mathbb{N}$:

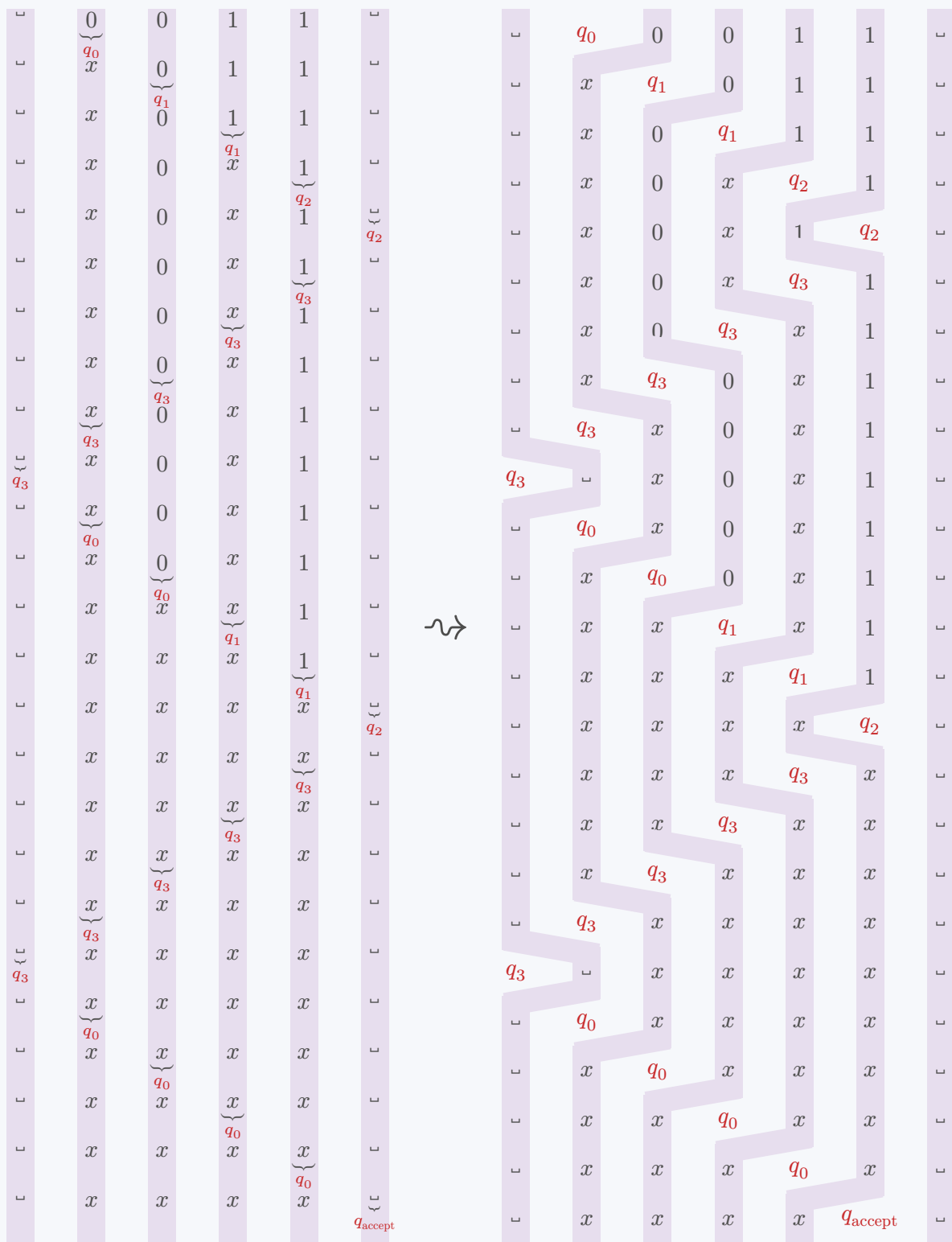
- 1) $A \cup B, \bigcup_k A_k$ abzählbar unendlich
- 2) $A \times B$ abzählbar unendlich
- 3) Abzählbar unendlich: $\mathbb{N}, \mathbb{Z}, \mathbb{Q}$
- 4) $P(A)$ überabzählbar unendlich
- 5) $\mathbb{R}$ überabzählbar unendlich

Beispiel 16 : $\{0^n 1^n \mid n \geq 0\}$ **10.2. PROTOKOLLIERUNG**

Der Gang der Berechnung mit einer TM kann dokumentiert werden, indem der Bandinhalt nach jedem einzelnen Verarbeitungsschritt protokolliert wird.

Links**Rechts**

Beispiel 17: $L = \{0^n 1^n \mid n \geq 0\}, w = 0011$



10.3. VARIANTEN

Laufzeit: Die Laufzeit einer TM M bei der Verarbeitung eines Wortes $w \in \Sigma^*$ ist die Anzahl $t(w)$ der Schritte, die die Turing-Maschine ausführt, bis sie anhält.

10.3.1. Nichtdeterministische TM

Übergangsfunktion

Bei jedem Übergang maximal N verschiedene Möglichkeiten: $\delta : Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{L, R\})$

$\Rightarrow$ Mehrere mögliche Berechnungswege

Wort akzeptieren

$w \in L(M)$ wenn es einen Berechnungsweg gibt, der zu q_{accept} führt. (auch wenn es Wege gibt, die auf q_{reject} führen!)

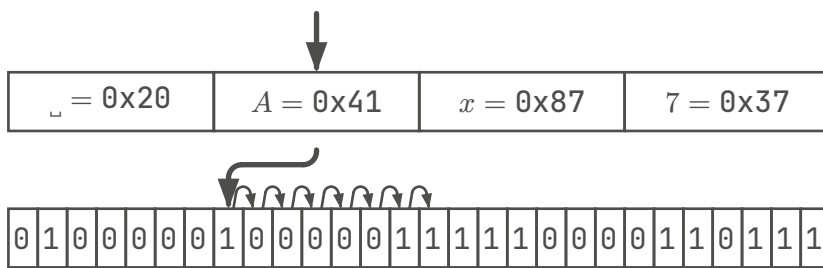
Simulationenidee

Alle Berechnungswege durchführen, maximal $N^{t(n)}$

Simulierbar in $O(N^{t(n)}) = 2^{O(t(n))}$

10.3.2. Verschiedene Bandalphabete

Jedes andere Alphabet als $\Sigma = \{0, 1\}$ kann binär codiert werden.



Simulierbar in Zeit $O(t(n))$

10.3.3. Mehrspurmaschine

TODO:

buch ..250.., slides 14

Simulierbar in Zeit $O(t(n))$

10.3.4. Mehrbandmaschine

Um eine Mehrbandmaschine mit n Bändern auf einer einfacheren Maschine zu simulieren, kann die unabhängige Bewegung der Schreib-/Leseköpfe mithilfe eines $2n$ -spurigen Bandes (Daten + Zeiger für Schreib-/Lesekopf) nachgebildet werden.

TODO:

buch ..250..

Simulierbar in Zeit $O(t(n)^2)$

10.3.4.1. Harvard- und von Neumann-Architektur

Die AVR-Mikrocontroller-Familie von Atmel verwendet intern drei unabhängige Datenpfade:

- Flash-Speicher, der den Programmcode enthält
- Statisches RAM, welches zur Laufzeit veränderliche Daten speichert
- Peripheriegeräte.

Die von Neumann-Architektur vereinheitlicht Daten- und Programmspeicher.

Simulation auf Standard-TM

Verwende 3 Bänder:

- 1) Arbeitsband
- 2) Kopie von w
- 3) Liste aller Folgen von Wahlmöglichkeiten

Simulation

- 1) Kopiere w von Band 2 auf Band 1
- 2) Führe TM aus auf Band 1 mit Wahlmöglichkeiten von Band 3
- 3) Inkrementiere zur nächsten Folge von Wahlmöglichkeiten auf Band 3

10.3.5. Einseitig unendliches Band

Beidseitig unendliches Band kann durch ein Alphabet doppelter Wortbreite realisiert werden, somit äquivalent.

TODO:

diagram (buch 235)

10.4. TURING-ERKENNBARE SPRACHEN

Eine TM erkennt das Wort $w \in \Sigma^*$, wenn die Maschine auf dem Input w im Zustand q_{accept} anhält.

Sind L_1 und L_2 Turing-erkennbare Sprachen, dann sind auch der Durchschnitt $L_1 \cap L_2$ und die Vereinigungsmenge $L_1 \cup L_2$ Turing-erkennbar.

Es gibt überabzählbare viele Sprachen $L \subset \Sigma^*$, die **nicht** Turing-erkennbar sind.

10.4.1. Aufzähler

Aufzähler: TM mit einem Drucker, mit dem Wörter ausgedruckt werden können.

L ist **rekursiv aufzählbar** wenn es einen Aufzähler gibt, der alle Wörter aufzählen kann. L ist dann auch Turing-erkennbar.

10.4.2. Entscheider und entscheidbare Sprachen

Turing-erkennbare Sprache: L heisst **Turing-erkennbar**, wenn es eine TM M gibt mit $L = L(M)$.

Turing-entscheidbare Sprache: L heisst **Turing-entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$.

Berechenbare Funktion: Eine Funktion $f : \Sigma^* \rightarrow \Sigma^*$ heisst **berechenbar**, wenn es eine TM M gibt, die auf jedem Inputwort $w \in \Sigma^*$ anhält und auf dem Band das Outputwort $f(w)$ zurücklässt.

TODO:

nicht entscheidbare Probleme

11. ENTSCHEIDBARKEIT

«Die sich mit immer neuen Superlativen gegenseitig überbietenden Ankündigungen der IT-Industrie könnten den Eindruck erwecken, dass mit geeignet viel Rechenleistung und Speicherplatz kein Informatikproblem einer Lösung widerstehen könnte.»

— Prof. Dr. Andreas Müller, [\[1, p. 267\]](#)

> based

TODO:

(Zehntes hilbertsches Problem). Man gebe ein Verfahren an, welches für eine beliebige diophantische Gleichung entscheidet, ob sie lösbar ist.

Serialisierung ermöglicht, jedes Entscheidungsproblem auf die Verarbeitung einer Zeichenkette zurückzuführen, die entweder akzeptiert oder verworfen wird. Jedes Problem wird auf diese Weise zu einem Sprachproblem.

11.1. ENTSCHEIDER

Definition

Ein **Entscheider** ist eine Turing-Maschine, die auf jedem beliebigen Input anhält.

Eine Sprache L heisst **entscheidbar**, wenn es einen Entscheider M gibt mit $L = L(M)$. Man sagt, M entscheidet L .

Sprachproblem

Jedes Problem P kann in ein Sprachproblem übersetzt werden:

$$L_P = \{w \in \Sigma^* \mid w \text{ ist Lösung des Problems } P\}$$

Beispiele

Leerheitsproblem (Entscheidbar):

$$E_{\text{DEA}} = \{\langle A \rangle \mid A \text{ ein DEA und } L(A) = \emptyset\}$$

Gleichheitsproblem (Entscheidbar):

$$\text{EQ}_{\text{CFG}} = \{\langle G_1, G_2 \rangle \mid G_i \text{ CFGs und } L(G_1) = L(G_2)\}$$

Akzeptanzproblem (Entscheidbar – nicht für TM!):

$$A_{\text{DEA}} = \{\langle A, w \rangle \mid A \text{ ein DEA, der } w \text{ akzeptiert}\}$$

Halteproblem (Nicht entscheidbar):

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ hält auf Input } w\}$$

11.2. AKZEPTANZPROBLEM FÜR TURING-MASCHINEN

$$A_{\text{TM}} = \{\langle M, w \rangle \mid M \text{ ist eine TM, die das Wort } w \in \Sigma^* \text{ akzeptiert}\}$$

Theorem

A_{TM} ist nicht entscheidbar.

Beweis

Konstruiere aus einem Entscheider H für A_{TM} eine Maschine D mit Input $\langle M \rangle$

- 1) Lasse H auf Input $\langle M, \langle M \rangle \rangle$
- 2) Falls H akzeptiert: q_{reject}
- 3) Falls H verwirft: q_{accept}

Wende jetzt D auf $\langle D \rangle$ an:

$$\begin{aligned} D(\langle D \rangle) \text{ akzeptiert} &\Rightarrow D \text{ verwirft } \langle D \rangle \\ D(\langle D \rangle) \text{ verwirft} &\Rightarrow D \text{ akzeptiert } \langle D \rangle \end{aligned}$$

11.3. HALTEPROBLEM

$$\text{HALT}_{\text{TM}} = \{\langle M, w \rangle \mid \text{die TM } M \text{ hält auf dem Input } w\}$$

$\Rightarrow$ Gleicher Widerspruch wie beim Akzeptanzproblem. Umschreiben des Problems in ein neues Programm S , welches «Akzeptieren» in «Anhalten» übersetzen soll = Reduktion ($A_{\text{TM}} \leq \text{HALT}_{\text{TM}}$)

TODO:

spezielles/allgemeines Halteproblem

11.4. REDUKTION

Reduktionsabbildung

Berechenbare Abbildung $f : \Sigma^* \rightarrow \Sigma^*$ so, dass

$$w \in A \Leftrightarrow f(w) \in B$$

Notation: $f : A \leq B$ heisst « A leichter entscheidbar als B »

Entscheidbarkeit

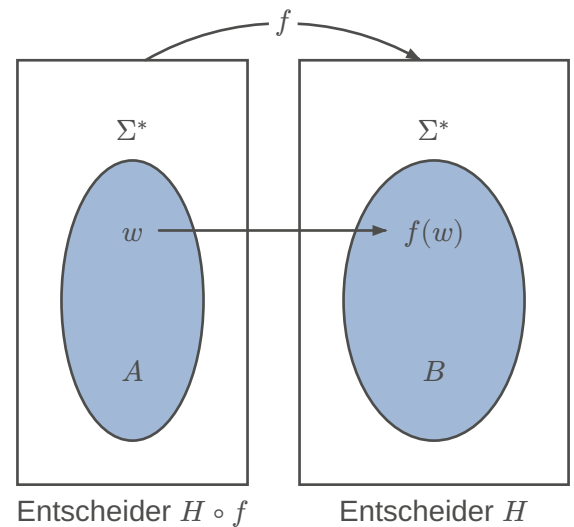
Falls B entscheidbar, dann $f : A \leq B \Rightarrow A$ entscheidbar

Beweis

Ist H ein Entscheider für B , dann ist $H \circ f$ ein Entscheider für A .

Folgerung

A nicht entscheidbar, $A \leq B \Rightarrow B$ nicht entscheidbar



11.5. VERGLEICH DER ENTSCHEIDBARKEIT VON SPRACHEN

Beispiel 18

TODO:

11.6. SATZ VON RICE

11.7. LÖSUNGSMETHODEN FÜR ENTSCHEIDUNGSPROBLEME

TODO:

book 273..

12. KOMPLEXITÄT

Ineffizient: Entscheidbare Probleme mit exorbitant langer Laufzeit sind praktisch «nicht lösbar»

Problemgrösse: Laufzeit hängt von der Problemgrösse n ab

Worst case Laufzeit: Die worst case Laufzeit $t(n)$, die die Maschine zur Verarbeitung von Inputs der Länge n braucht, ist $t(n) = \max\{t(w) \mid |w| \leq n\}$

12.1. HARDWAREEINFLUSS

Eine Sprache L gehört zur Klasse NP , wenn L von einer *nicht*deterministischen TM in **polynomieller Zeit** entschieden werden kann, $t(n) = O(n^k)$

TODO:

slides 3,4

12.2. POLYNOMIELLE UND EXPONENTIELLE LAUFZEIT

TODO:

slides 5

<i>Polynomielle Probleme</i>	<i>Exponentielle Probleme</i>
– Gauss-Algorithmus $O(n^3)$	– Faktorisierung von grossen Zahlen (RSA)
– ...	– ...
Skalierbarkeit	Sicherheit

12.3. HARDWAREUNABHÄNGIGE LAUFZEIT-KOMPLEXITÄT

Ein Algorithmus hat **polynomielle Laufzeit**, wenn seine Laufzeit durch ein Polynom beschränkt ist: $t(n) = O(n^k)$.

12.4. NP

Eine Sprache gehört zur Klasse **NP**, wenn sie von einem **nichtdeterministischen** Entscheider in polynomieller Zeit entschieden werden kann

12.5. POLYNOMIELLE VERIFIZIERER

Ein **polynomieller Verifizierer** für die Sprache L ist eine TM V , so dass es für jedes $w \in \Sigma^*$ ein Wort c (das Lösungszertifikat) gibt, für das gilt

$$w \in L \Leftrightarrow V \text{ akzeptiert } \langle w, c \rangle$$

Die Laufzeit von V ist polynomiell in $|w|$.

Eine Sprache ist genau dann in **NP**, wenn sie in polynomieller Zeit **verifiziert** werden kann.

TODO:

slides 14

12.6. POLYNOMIELLE REDUKTION

12.6.1. Polynomieller Laufzeit-Vergleich

Seien A und B entscheidbare Sprachen. Eine berechenbare Abbildung

$$f : \Sigma^* \rightarrow \Sigma^* : w \mapsto f(w)$$

mit

- 1) $w \in A \Leftrightarrow f(w) \in B$ (Reduktion)
- 2) $f(w)$ ist berechenbar in polynomieller Zeit in $|w|$

heisst **polynomielle Reduktion** $A \leq_P B$. Lies: A ist polynomiell leichter entscheidbar als B .

Polynomiell äquivalent: Zwei Sprachen A und B heißen **polynomiell äquivalent**, geschrieben $A \equiv_P B$, wenn $A \leq_P B$ und $B \leq_P A$.

12.6.2. Satz

Sei $A \leq_P B$. Dann gilt

$$\begin{aligned}
 B \in P &\Rightarrow A \in P, & A \notin P &\Rightarrow B \notin P \\
 B \in NP &\Rightarrow A \in NP, & A \notin NP &\Rightarrow B \notin NP
 \end{aligned}$$

TODO:

slides 16,17

12.6.3. Reduktion von HAMPATH**TODO:****12.7. POLYNOMIELLE AUSFÜLLRÄTSEL****Definition****Variablen**

Ein polynomielles Ausfüllrätsel ist eine $n \times m$ -Tabelle, in die Zeichen eines Alphabets Σ eingefüllt werden müssen, so dass als logische Formel ausdrückbare Regeln eingehalten werden, die in polynomieller Zeit ausgewertet werden können.

Genau ein Zeichen im Feld (i, j)

$$\varphi_{ij} = \bigvee_{c \in \Sigma} \left(\underbrace{x_{ijc} \wedge \bigvee_{d \neq c} x_{ijd}}_{c \text{ und kein anderes Zeichen in } (i,j)} \right)$$

Satz

Jedes polynomielle Ausfüllrätsel A lässt sich polynomiell auf SAT reduzieren.

Regeln

Spielregeln als Formel φ in Variablen x_{ijc} ausdrücken, die wahr ist genau dann, wenn die Regeln erfüllt sind.

Beispiel

$n^2 \times n^2$ – Sudoku

BIBLIOGRAFIE

- [1] A. Müller, «Reguläre Sprachen», in *Automaten und Sprachen: Theoretische Informatik für die Praxis: Mathematik, Anwendung, Intuition*, Berlin, Heidelberg: Springer Berlin Heidelberg, 2024.
doi: [10.1007/978-3-662-70146-1_1](https://doi.org/10.1007/978-3-662-70146-1_1) .